

IEEE Solid-State Circuits Society (SSCS) — Egypt Chapter

2026 Student Design Competition

FPGA-Based Edge-AI Vision Accelerator

N×N CNN Convolution Accelerator — RTL Design, Verification & Implementation Report

Prepared by:

Group: The Bit Spies

Team Members:

Saif Eldeen Fathi Ali

Ahmed Amr Farouk

Toka Abdelhamed Mohamed

Alaa Kareem Abdelmawla

Institution: Capital University / Helwan University — Electronics & Communication Engineering

Submission date: September 25, 2026

Table of Contents

1. Introduction.....	4
2. Specification Compliance Summary.....	4
3. Requirement-by-Requirement Mapping	5
4. Architecture Overview.....	6
4.1 Module list	7
4.2 Control FSM	8
5. Fixed-Point Bit-Width Analysis	9
6. Pipeline / Timing Behaviour	9
7. Verification Methodology.....	10
8. FPGA Implementation Results	11
8.1 Resource utilization	11
8.2 Timing.....	12
8.3 Power	12
9. Figure of Merit (FoM)	13
10. Design Tradeoffs and Final Notes	14
10.1 Tradeoffs.....	14
10.2 Design Limitations and Final Notes.....	14
11. Deliverables	15

List of Figures

Figure 1: System-Level Block Diagram	6
Figure 2: RTL Schematic (Vivado)	6
Figure 3: RTL Schematic Post-Synthesis	7
Figure 4 FSM States.....	8
Figure 5:Post-Implementation Resource Utilization (Vivado).....	11
Figure 6:Design Timing Summary — All Constraints Met	12
Figure 7: On-Chip Power Summary (Vivado, Without SAIF Activity Annotation).....	12
Figure 8:On-Chip Power Summary (Vivado, SAIF Activity Applied).....	13

List of Tables

Table 1:Specification Compliance Summary	4
Table 2:Requirement-by-Requirement Mapping.....	5
Table 3:RTL Module List	7
Table 4:Fixed-Point Bit-Width Analysis	9
Table 5: Regression Test Cases.....	10
Table 6:Power-Analysis Comparison	13
Table 7:Deliverables	15

1. Introduction

This report is submitted for the IEEE SCS Egypt Chapter 2026 Student Design Competition, which calls for an FPGA-based Edge-AI vision accelerator performing $N \times N$ CNN convolution on a grayscale image or feature map. It documents the accelerator's architecture, RTL implementation, fixed-point design, verification methodology, and real FPGA implementation results (synthesis, timing, and power) obtained from Vivado, together with the resulting Figure of Merit (FoM).

2. Specification Compliance Summary

Table 1: Specification Compliance Summary

Parameter	Specification	Team Result	Units	Comments
Input image size	$\geq 32 \times 32$	32×32	pixels	IMAGE_SIZE = 32 (parameter); single-channel
Input precision	Unsigned fixed-point	8-bit unsigned	bits	DATA_WIDTH = 8
Kernel precision	8-bit signed	8-bit signed	bits	Two's-complement, loaded via weight in
Architecture type	—	Pipelined parallel PE array, line-buffer based	—	Streaming, valid convolution
Multipliers / MACs	—	$9 (N^2 \text{ for } N = 3)$	DSP48	One PE per kernel tap
Pipeline stages	—	Registered PE + pipelined adder tree	—	See Section 6
Latency	—	72	cycles	Measured RTL-simulation latency from start at 65 ns to first output_valid at 785 ns with a 10 ns simulation clock period
Throughput	px/cycle (per spec)	Peak = 1.0; average ≈ 0.939	output px/cycle	Average sustained basis: 900 / 958; 58 gap cycles occur at 29 row boundaries; one-output-pixel-per-cycle bonus is not claimed
FPGA utilization	—	LUT 491, LUTRAM 1, FF 902, DSP 9, IO 43	—	Vivado post-implementation results, Section 8
Operating / closed frequency	—	209.8	MHz	≈ 4.766 ns period; timing closed with WNS = +0.013 ns
Power estimate	—	0.163	W	Vivado post-implementation total on-chip power after SAIF activity annotation
Verification status	—	Baseline PASS; 14-case regression configured	—	Python/NumPy golden model + self-checking SystemVerilog testbench
FOM	Throughput / (Power $\times$ (LUTs + 50 $\cdot$ DSPs + 100 $\cdot$ BRAMs))	$\approx 6.12 \times 10^{-3}$	(px/cycle)/(W $\cdot$ resource-unit)	See Section 9.

3. Requirement-by-Requirement Mapping

Table 2: Requirement-by-Requirement Mapping

#	Requirement	Implementation
1	Min. 32×32 input, single-channel	IMAGE_SIZE = 32 (parameterizable in cnn_config.vh); pixel_in is a single 8-bit stream
2	Unsigned fixed-point input	8-bit unsigned pixel_in (Q8.0); justified in Section 5
3	Programmable N×N kernel	reg_kernel serially loads N ² =9 signed coefficients via weight_in / weight_valid
4	8-bit signed kernel	DATA_WIDTH = 8, two's-complement weights
5	Stride = 1	line_buff advances the sliding window by exactly one column per valid pixel
6	Output ≥16-bit signed, overflow handling	OUTPUT_WIDTH = 2·DATA_WIDTH + ⌈log ₂ (K ²)⌉ = 20 bits, wide enough that the 3×3/8-bit dot product never overflows; ReLU stage saturates negative results to zero
7	Optional ReLU (bonus)	Dedicated ReLU module: f(x) = max(0, x), gated by st_ReLU from the FSM
8	Verification vs. golden model	Python/NumPy golden_model.py + self-checking SystemVerilog testbench (tb_cnn_top.sv), bit-exact compare (Section 7)
9	FPGA synthesis/implementation results	Real Vivado synthesis + implementation results reported in Section 8 (LUT/FF/DSP/IO, timing, power)
10	Figure of Merit	Computed in Section 9 using LUT count (FFs excluded), weighted DSP/BRAM terms, average throughput, and SAIF-based total post-implementation power
Bonus	ReLU activation	Implemented (see #7)

4. Architecture Overview

The accelerator streams input pixels into `line_buff`, which forms 3×3 sliding windows. Each valid window feeds nine parallel PEs (one per kernel tap), and their signed products are reduced by a balanced pipelined adder tree in `PE_Array`. The 3×3 kernel is serially loaded into `reg_kernel` before convolution. ReLU clamps negative outputs to zero, while `control_fsm` sequences kernel loading and the convolution/streaming phase.

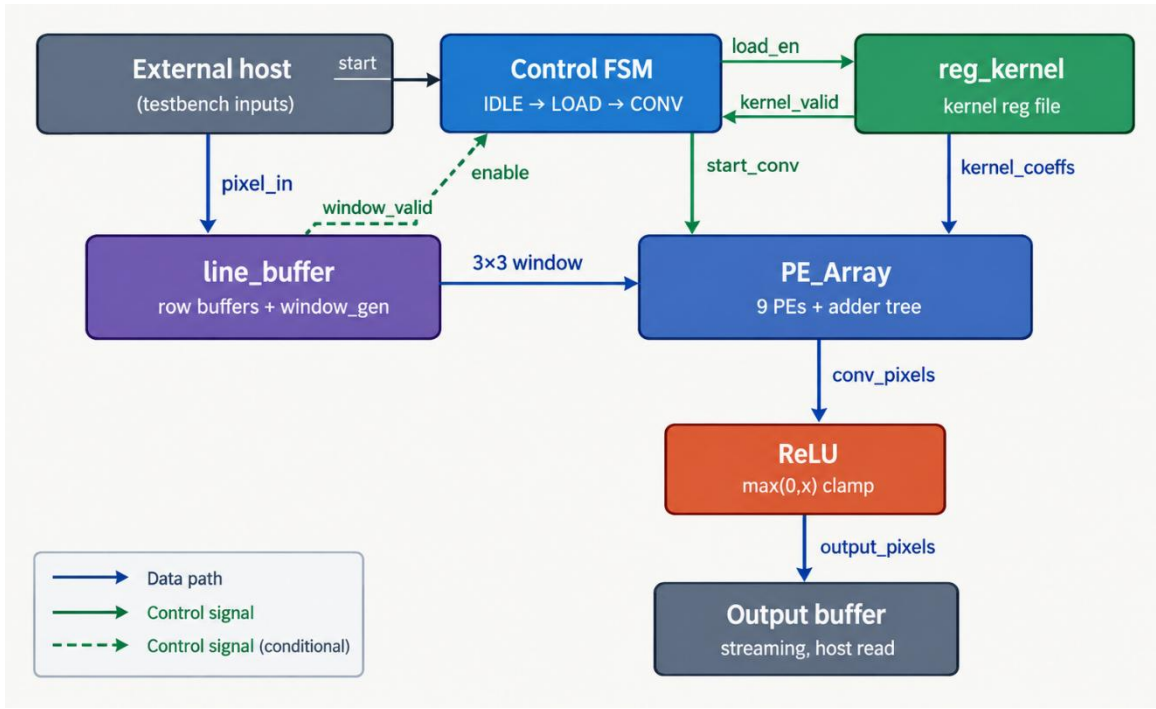


Figure 1: System-Level Block Diagram

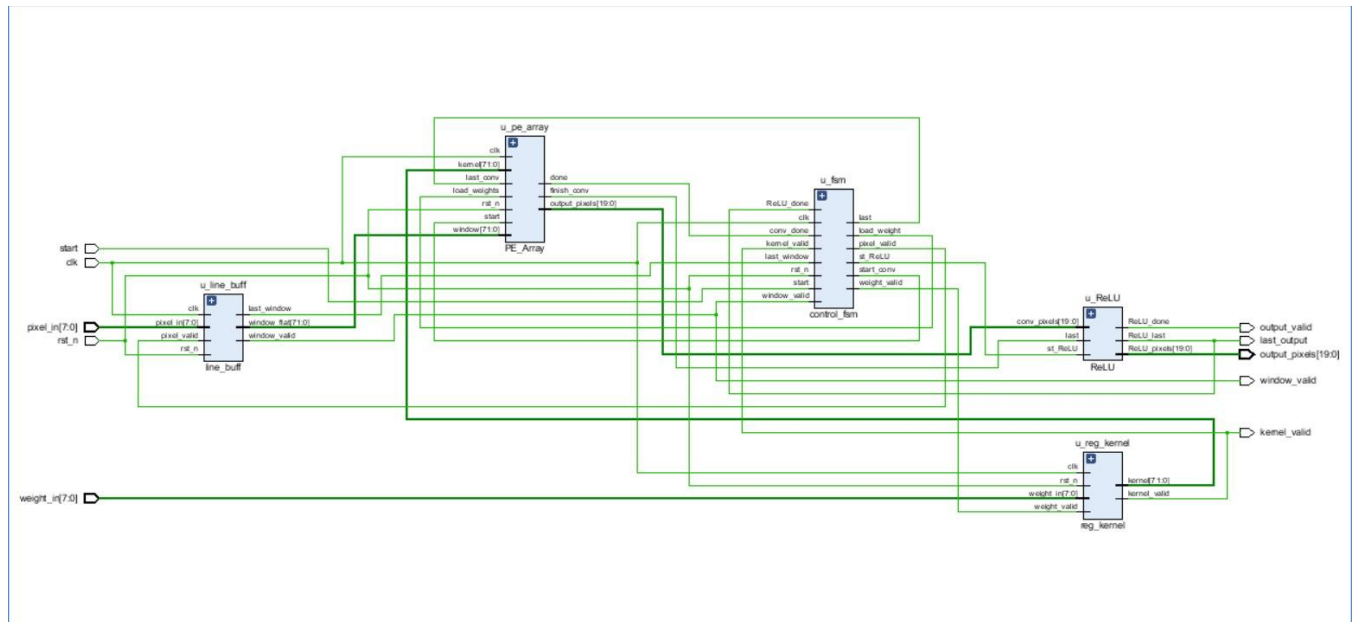


Figure 2: RTL Schematic (Vivado)

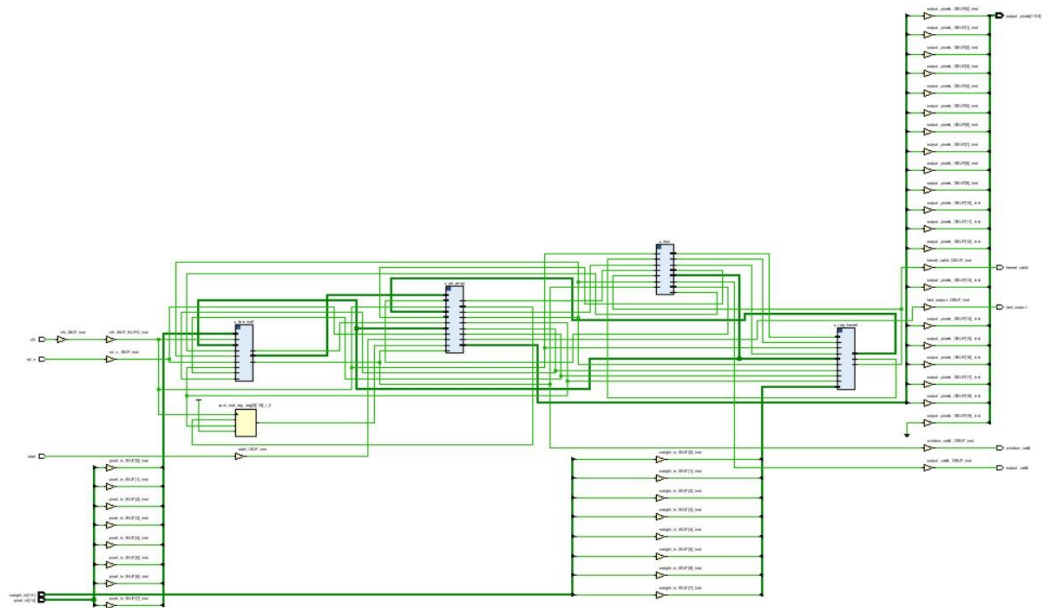


Figure 3: RTL Schematic Post-Synthesis

4.1 Module list

Table 3: RTL Module List

Module	File	Role
cnn_top	cnn_top.sv	Top-level integration; wires all submodules together
control_fsm	FSM.sv	3-state control FSM: IDLE → LOAD → CONV
reg_kernel	REG_Kernel.sv	Serially-loaded 3×3 signed kernel coefficient register file
line_buff	line_buff.sv	Row-delay line buffers + 3×3 sliding-window (window_gen) generation
PE	PE.sv	Single processing element: unsigned pixel is zero-extended and multiplied by an 8-bit signed weight using a native signed multiply with use_dsp="yes"; the 16-bit product is captured in an output register when start is asserted.
PE_Array	PE_Array.sv	Nine registered PEs plus a balanced adder tree. The tree is split at MID_LEVEL with a midpoint pipeline register, then the remaining levels are reduced and registered to output_pixels; pip_done/pip_last align done and finish_conv.
ReLU	ReLU.sv	Optional ReLU post-processing stage; also gates last_output

4.2 Control FSM

The FSM (FSM.sv) has three states:

- **IDLE** — waits for start; all datapath enables de-asserted.
- **LOAD** — streams `weight_in` into `reg_kernel` (`weight_valid` high) while simultaneously beginning to stream the first image rows into `line_buff` so the first window is ready the moment the kernel finishes loading (`kernel_valid`).
- **CONV** — main streaming/compute phase; `start_conv` follows `window_valid`, and `st_ReLU` is asserted once `conv_done` indicates a valid convolution result; `last` is asserted on the `last_window` pulse and propagated through `ReLU_last` back to the FSM as `ReLU_done`, returning the machine to IDLE.

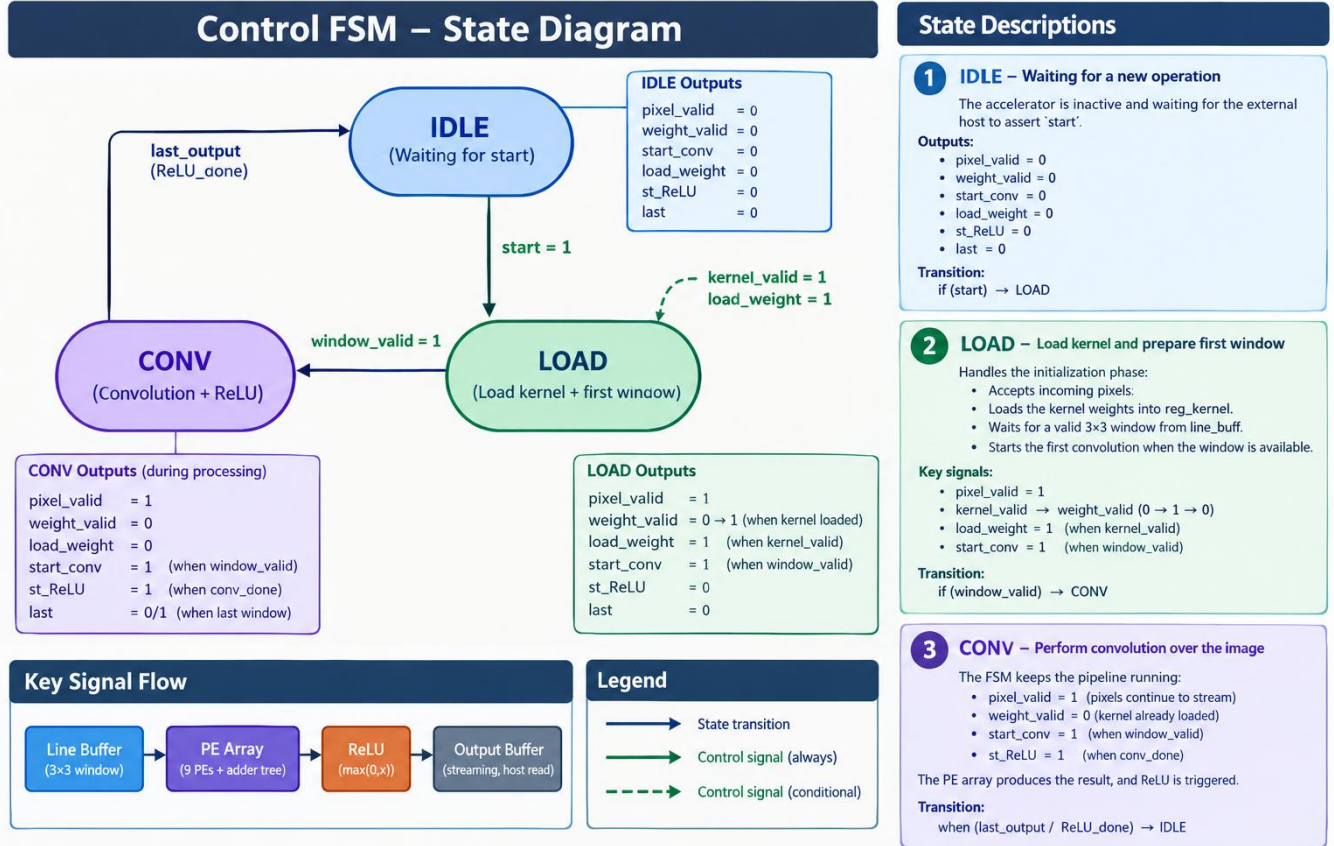


Figure 4 FSM States

5. Fixed-Point Bit-Width Analysis

Table 4: Fixed-Point Bit-Width Analysis

Signal	Format	Width	Justification
Input pixel (pixel_in)	Unsigned (Q8.0)	8 bits	Standard 8-bit grayscale depth; matches spec #2, resource-efficient for an edge accelerator
Kernel coefficient (weight_in)	Signed, two's-complement	8 bits	Mandated by spec #4
PE product (A×B)	Signed	16 bits	$\text{DATA_WIDTH} \times 2$ = minimum width that cannot overflow an 8-bit unsigned × 8-bit signed product
Convolution output (conv_pixels / output_pixels)	Signed	20 bits ($2 \cdot \text{DATA_WIDTH} + \lceil \log_2(K^2) \rceil = 16+4$)	Sized with guard bits so the full 9-term dot product of full-range operands cannot overflow; exceeds the mandated 16-bit minimum (spec #6)
ReLU output	Signed	20 bits ($2 \cdot \text{DATA_WIDTH} + \lceil \log_2(K^2) \rceil = 16+4$)	$f(x) = \max(0, x)$; negative results are zeroed rather than saturated/wrapped, since the sign bit alone identifies overflow-free negative results in this format

Because OUTPUT_WIDTH is derived symbolically from DATA_WIDTH and KERNEL_SIZE, changing the kernel size in cnn_config.vh automatically keeps the accumulator wide enough to avoid overflow, so no separate saturation logic was required in the convolution path for the 3×3 configuration; only the ReLU stage performs a data-dependent clamp (to zero).

6. Pipeline / Timing Behaviour

The current PE.sv implements a native signed multiply intended for DSP48 inference. The unsigned pixel operand A is zero-extended by one bit ($A_ext = \{1'b0, A\}$) and then multiplied by the signed 8-bit kernel coefficient B. The 16-bit product is captured directly in register C on a rising clock edge when start is asserted. Therefore, the current PE source no longer contains the previously described separate post-multiply and output register stages.

PE_Array.sv instantiates $N_PE = \text{KERNEL_SIZE}^2$ PEs (nine for the 3×3 configuration). The PE outputs are sign-extended to OUTPUT_WIDTH and reduced by a balanced adder tree. To shorten the combinational critical path, the tree is divided at MID_LEVEL and its partial sums are captured in sum_mid_reg before the remaining adder levels. The final sum is then registered into output_pixels. The start/last_conv validity information is propagated through pip_done and pip_last so that done and finish_conv track the pipelined datapath. This midpoint register is the key timing-oriented change in the supplied PE_Array RTL.

Latency and throughput. RTL simulation measures 72 clock cycles from start to the first valid output: start occurs at 65 ns and the first output_valid occurs at 785 ns with a 10 ns simulation clock period. After the pipeline is filled, each 30-pixel output row is produced at one valid output per cycle, followed by a two-cycle gap before the next output row. For the 30×30 output feature map, there are 900 valid outputs and 29 row boundaries, giving $29 \times 2 = 58$ gap cycles. Therefore, the average sustained throughput is $900 / (900 + 58) = 900 / 958 \approx 0.9395$ output pixels/cycle. The one-output-pixel-per-cycle bonus is not claimed because output_valid is not continuously asserted across row boundaries. Output pixels are produced in row-major order: left to right within each 30-pixel output row, then top to bottom across the 30×30 output feature map. Each asserted output_valid corresponds to one spatial convolution result.

7. Verification Methodology

Golden Reference Model. The Python/NumPy `golden_model.py` is an independent reference implementation of the same 3×3 valid, stride-1 CNN cross-correlation used by the RTL. The model uses unsigned 8-bit input pixels, signed 8-bit kernel coefficients, a fixed reproducible seed (42) for generated random data, optional ReLU enabled for the exported reference set, and explicit output overflow handling. It exports \$readmemh-compatible `kernel.mem`, `image.mem`, and `relu_output.mem` files. The exported golden output file is 16-bit signed; when the RTL output bus is wider, the testbench sign-extends the reference value before comparison.

Self-checking RTL verification. `tb_cnn_top.sv` loads the stimulus and expected-output files, streams the complete 32×32 image and programmable 3×3 kernel into the DUT, and compares every valid hardware output against the corresponding Python reference value. Each complete case produces a 30×30 output map, i.e. 900 expected values. The current regression set contains 14 full-image cases (12,600 output comparisons if all cases are run). The original baseline case has been observed to match all 900 golden outputs; the extended suite is structured for automatic per-case PASS/FAIL reporting.

Table 5: Regression Test Cases

#	Case	Input / Kernel	Coverage purpose
1	original_primary	Seeded random image + seeded quantized random kernel	Baseline full-frame functional check
2	vectors_zeros	All-zero image + primary kernel	Zero-input edge case
3	vectors_max_value	All pixels = 255 + primary kernel	Full-scale input / output-range stress
4	vectors_checkerboard	Checkerboard image + primary kernel	Alternating high-activity spatial pattern
5	vectors_ramp	Horizontal ramp image + primary kernel	Monotonic gradient / window sequencing
6	showcase_identity	Showcase image + identity kernel	Pixel/window ordering and basic convolution
7	showcase_edge_h	Showcase image + Sobel-X kernel	Signed weights; vertical-edge response
8	showcase_edge_v	Showcase image + Sobel-Y kernel	Signed weights; horizontal-edge response
9	showcase_sharpen	Showcase image + sharpen kernel	Mixed positive/negative coefficients
10	showcase_box_blur	Showcase image + 3×3 all-ones kernel	Nine-term accumulation
11	checkerboard_box_blur	Checkerboard image + box-blur/sum kernel	Repeated high-frequency pattern + accumulation
12	ramp_edge_h	Ramp image + Sobel-X kernel	Predictable gradient edge response
13	max_value_sharpen	Full-scale image + sharpen kernel	Extreme-value / saturation stress
14	random_edge_v	Seeded random image + Sobel-Y kernel	General non-structured signed-kernel regression

Verification evidence note. The testbench reports mismatch counts and case-level PASS/FAIL status. The full 14-case transcript should be retained with the final submission as verification evidence.

Simulation run scripts. Two QuestaSim .do scripts are used: **run_multi_cases.do** executes the complete self-checking 14-case regression and produces the functional-verification transcript, while **run_saif.do** captures switching activity during active processing and exports convolution.saif for Vivado post-implementation power analysis. The two flows are kept separate for functional verification and SAIF generation.

```

# =====
# CASE [test_cases/random_edge_v]: captured 900 samples, expected 900
# CASE [test_cases/random_edge_v][          14]: PASSED (alignment offset = 0)
# =====
#####
# FINAL SUMMARY: 14 total, 14 passed, 0 failed
# ALL 14 TEST CASES PASSED
#####
# ** Note: $stop      : tb_cnn_top.sv(264)
#    Time: 149235 ns  Iteration: 1  Instance: /tb_cnn_top
# Break in Module tb_cnn_top at tb_cnn_top.sv line 264

```

8. FPGA Implementation Results

The design was synthesized and implemented in Vivado, targeting the PYNQ-Z2 board (XC7Z020-1CLG400C), at an operating frequency of 209.8 MHz (approximately 4.766 ns clock period). The figures below are the latest post-implementation results. Resource utilization and timing are taken from the implemented design. For the final power figure, switching activity from the generated SAIF file was applied before running Vivado power analysis, so the reported dynamic power is activity-based.

8.1 Resource utilization

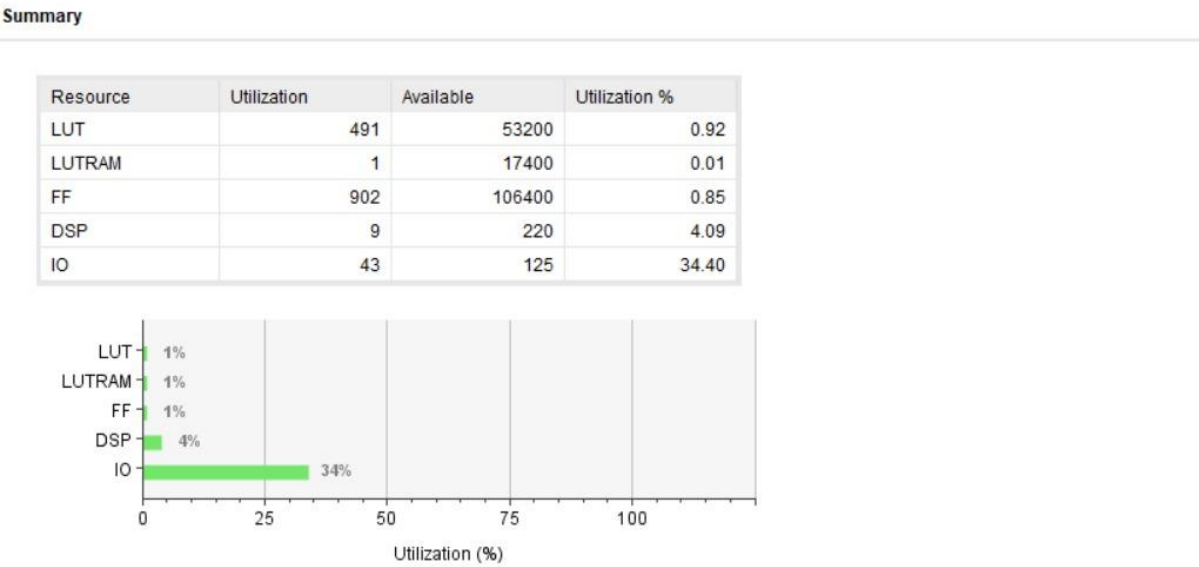


Figure 5:Post-Implementation Resource Utilization (Vivado)

8.2 Timing

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.013 ns	Worst Hold Slack (WHS): 0.162 ns	Worst Pulse Width Slack (WPWS): 1.402 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 1745	Total Number of Endpoints: 1745	Total Number of Endpoints: 913

All user specified timing constraints are met.

Figure 6: Design Timing Summary — All Constraints Met

- Worst Negative Slack (WNS): +0.013 ns — setup met, 0 failing endpoints out of 1,745
- Worst Hold Slack (WHS): +0.162 ns — hold met, 0 failing endpoints out of 1,745
- Worst Pulse Width Slack (WPWS): +1.402 ns — met, 0 failing endpoints out of 913
- All user-specified timing constraints are met at the 209.8 MHz operating frequency.

8.3 Power

Power-analysis sequence. Power was first estimated from an implemented-design power run without SAIF activity annotation, which reported 0.133 W total on-chip power. The analysis was then repeated with the generated SAIF switching activity applied, producing 0.163 W total on-chip power. The SAIF-based value is used in the final FoM calculation.

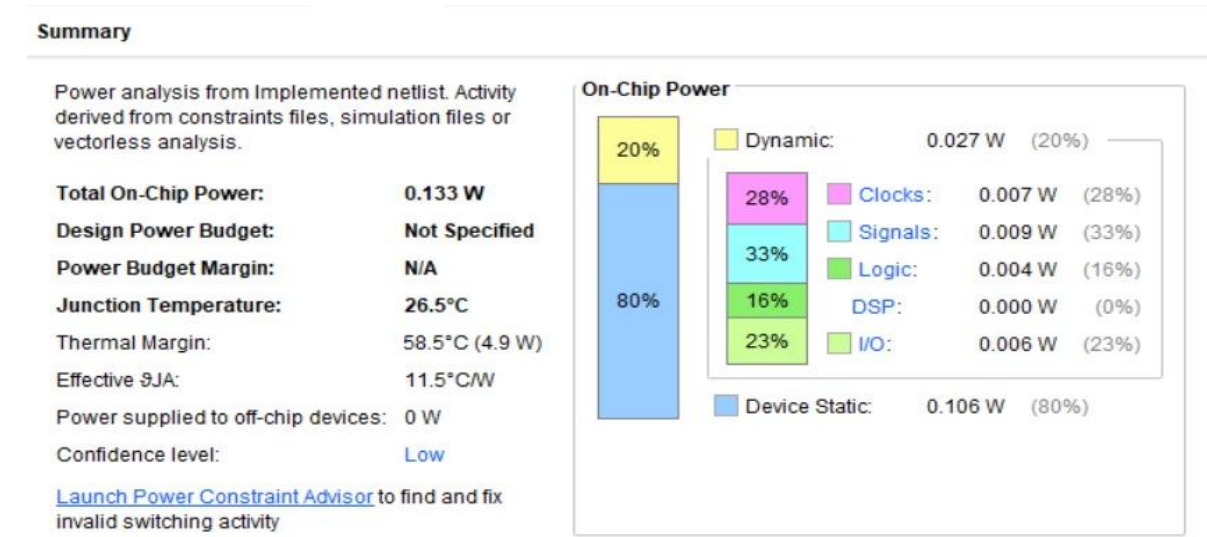


Figure 7: On-Chip Power Summary (Vivado, Without SAIF Activity Annotation)

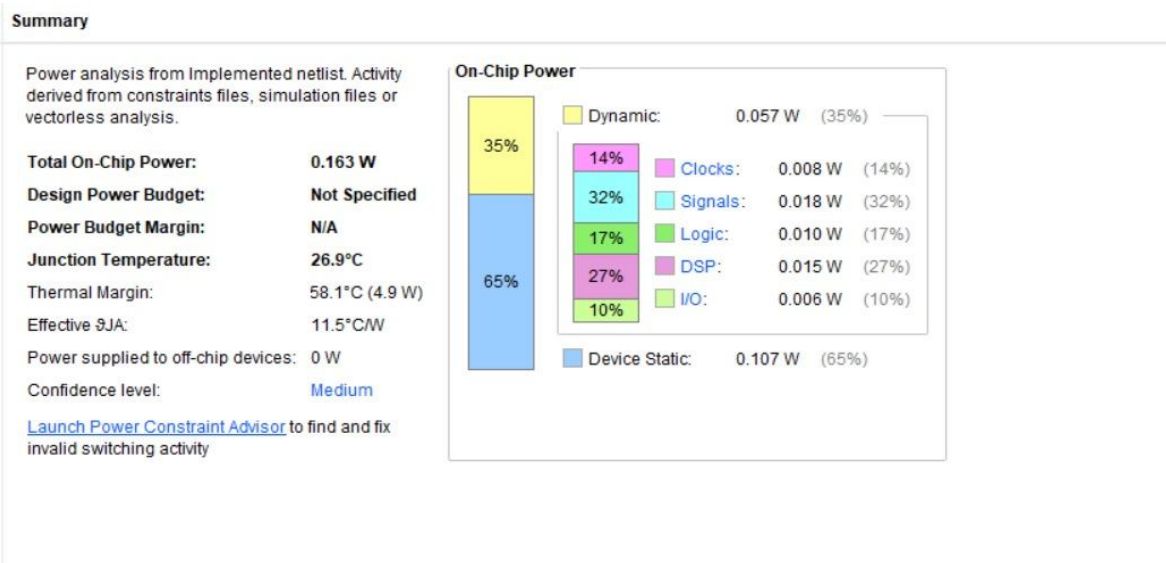


Figure 8: On-Chip Power Summary (Vivado, SAIF Activity Applied)

- Total on-chip power after SAIF activity annotation: 0.163 W
- Dynamic power: 0.057 W (35%) — Clocks 0.008 W, Signals 0.018 W, Logic 0.010 W, DSP 0.015 W, I/O 0.006 W.
- Device static power: 0.107 W (65%). Vivado reports a Medium confidence level for this power analysis.

Table 6: Power-Analysis Comparison

Quantity	Without SAIF	With SAIF
Dynamic power	0.027 W	0.057 W
Device static power	0.106 W	0.107 W
Total on-chip power	0.133 W	0.163 W

Comparison summary. Table 6 summarizes the measured power values from the earlier run without SAIF and the final SAIF-annotated run. The increase in dynamic power reflects the use of simulation-derived switching activity in the SAIF-based analysis.

SAIF / tool details. Activity was captured over the active convolution interval from approximately 75 ns to 10365 ns, excluding the long reset/idle region. Questa Sim 2021.1 was used to generate the switching activity, and Vivado post-implementation power analysis was used for the reported static, dynamic, and total power at the 209.8 MHz operating point.

9. Figure of Merit (FoM)

Per the competition formula, $FOM = \text{Throughput} / (\text{Power} \times (\text{LUTs} + 50 \cdot \text{DSPs} + 100 \cdot \text{BRAMs}))$. The official throughput unit is output pixels per cycle. Using the latest implementation resources (LUTs = 491, DSPs = 9, BRAMs = 0), the SAIF-based total post-implementation power of 0.163 W, and the measured average sustained throughput of $900/958 \approx 0.9395$ output pixels/cycle:

- LUT-equivalent denominator = $491 + 50 \times 9 + 100 \times 0 = 941$
- Power $\times$ denominator = $0.163 \times 941 = 153.383$
- Peak throughput = 1 valid output pixel/cycle during uninterrupted valid-window runs; this peak is not sustained across the entire frame, so the one-output-pixel-per-cycle bonus is not claimed.
- Average sustained throughput used for FoM = $900 / 958 \approx 0.9395$ output pixels/cycle
- Updated FOM (competition px/cycle units) = $0.9395 / 153.383 \approx 6.12 \times 10^{-3}$ (output px/cycle)/(W $\cdot$ resource-unit)

At 209.8 MHz, the same average sustained throughput corresponds to approximately 197.10 million output pixels/second. A frequency-scaled reference figure is therefore approximately 1.285×10^6 (output px/s)/(W $\cdot$ resource-unit). This frequency-scaled value is included only as a secondary reference; the competition FoM reported above uses output pixels per cycle.

10. Design Tradeoffs and Final Notes

10.1 Tradeoffs

- PE.sv now uses a native signed multiply with the unsigned pixel zero-extended by one bit and explicitly requests DSP implementation using the use_dsp attribute. The product is captured directly in C when start is asserted, simplifying the PE datapath while retaining one DSP per kernel tap.
- PE_Array.sv uses a balanced reduction tree rather than a long serial accumulation path. The tree is split at MID_LEVEL and the partial sums are registered in sum_mid_reg, reducing the amount of combinational addition between registers and helping the design close timing at the 209.8 MHz operating target.
- Output width is derived symbolically ($2 \cdot \text{DATA_WIDTH} + \lceil \log_2(K^2) \rceil$) instead of being hand-picked, so the design generalizes to other kernel sizes without a manual overflow re-analysis.

10.2 Design Limitations and Final Notes

- The one-output-pixel-per-cycle bonus is not claimed. Peak datapath throughput can reach 1 valid output/cycle within each output row, but output_valid contains two-cycle gaps at row boundaries; the measured average sustained throughput is ≈ 0.9395 px/cycle.
- The current FoM uses the measured average sustained throughput basis of $900/958 \approx 0.9395$ output pixels/cycle. The latency to the first valid output is measured as 72 clock cycles from the RTL waveform.
- The standard data path processes one 8-bit input pixel per cycle. Input streaming is controlled internally by the FSM through pixel_valid, while output_valid is asserted only when a valid convolution result is available.
- Retain the complete 14-case testbench transcript with mismatch counts and case-level PASS/FAIL results as submission evidence for the golden-model verification section.
- BRAM usage remains 0 in the reported implementation because the design streams pixels and uses line-buffer/register-based storage rather than a full-frame BRAM buffer.
- The final power/FoM result should continue to use the SAIF-annotated post-implementation total power (0.163 W) at the stated 209.8 MHz operating frequency.

11. Deliverables

Table 7: Deliverables

File	Description
cnn_config.vh	Global parameters: IMAGE_SIZE, KERNEL_SIZE, DATA_WIDTH, OUTPUT_WIDTH, AW
cnn_top.sv	Top-level integration module
FSM.sv	Control FSM (control_fsm)
REG_Kernel.sv	Programmable kernel coefficient register file (reg_kernel)
line_buff.sv	Row-delay line buffer / sliding-window generator
PE.sv	Registered DSP48-mapped signed multiply processing element
ReLU.sv	Optional ReLU post-processing stage
golden_model.py	Python/NumPy golden reference model and test-vector generator
tb_cnn_top.sv	Self-checking SystemVerilog testbench
PE_Array.sv	Nine-PE parallel convolution array with balanced pipelined adder tree
export_Test	Primary, edge-case vector, and multi-kernel golden-model .mem test files
Constraints_cnn_top.xdc	Pin assignment and 209.8 MHz operating clock constraint for the PYNQ-Z2 board